Make sure staticfiles is configured correctly

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Enable WhiteNoise

```
MIDDLEWARE = [
    # ...
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
    # ...
]
```

Add compression and caching support

```
STORAGES = {
    # ...
    "staticfiles": {
        "BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}
```

This combines automatic compression with the caching behaviour provided by Django's ManifestStaticFilesStorage backend. If you want to apply compression but don't want the caching behaviour then you can use the alternative backend:

```
STATICFILES_STORAGE = "whitenoise.storage.CompressedStaticFilesStorage"
```

Using WhiteNoise in development

```
INSTALLED_APPS = [
    # ...
    "whitenoise.runserver_nostatic",
    "django.contrib.staticfiles",
    # ...
]
```

How do I know it's working?

```
python manage.py collectstatic
```

Using WhiteNoise with Webpack / Browserify / $LATEST_JS_THING

```
STATICFILES_DIRS = [BASE_DIR / "static_build"]
```

তুমি যে configuration গুলো দেখিয়েছো সেগুলো **Django** project-এ **static files (CSS, JS, images)** production server-এ serve করার জন্য ব্যবহার করা হয়। এখানে **WhiteNoise** ব্যবহার করা হচ্ছে।

আমি একে একে সব term সহজভাবে explain করছি। 👇

1️⃣ `STATIC_ROOT`

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

কি কাজ করে

`collectstatic` run করলে সব static file **এক জায়গায় collect** হয়।

ধরো project structure:

```
project/
│
├── app1/
│   └── static/
│       └── css/style.css
│
├── app2/
│   └── static/
│       └── js/app.js
```

`collectstatic` run করলে:

```
staticfiles/
│
├── css/style.css
└── js/app.js
```

সব static file এক জায়গায় আসে।

Run command:

```
python manage.py collectstatic
```

Production server (Nginx / WhiteNoise) এখান থেকে file serve করে।

2️⃣ WhiteNoise middleware

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
]
```

কাজ

WhiteNoise middleware **static file directly serve করে**।

Normally production architecture হয়:

```
User
  ↓
Nginx
  ↓
Django
```

কিন্তু WhiteNoise থাকলে:

```
User
  ↓
Django + WhiteNoise
```

Django নিজেই static file serve করে।

সুবিধা

✔ আলাদা nginx লাগেনা
✔ deployment easy
✔ small project এর জন্য best

3️⃣ `STORAGES`

```
STORAGES = {
    "staticfiles": {
```

```
        "BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}
```

এটা **Django 4.2+** এর new configuration।

আগে ছিল:

```
STATICFILES_STORAGE
```

এখন replace হয়েছে `STORAGES` দিয়ে।

4️⃣ `CompressedManifestStaticFilesStorage`

```
whitenoise.storage.CompressedManifestStaticFilesStorage
```

এটা ২টা কাজ করে।

1️⃣ Compression

File compress করে:

```
style.css
style.css.gz
```

browser gzip support করলে `.gz` file serve হয় → faster loading.

2️⃣ Cache busting

Static file change হলে browser cache problem হয়।

Example:

```
style.css
```

User browser cache করে রাখে।

তুমি style change করলে user old style দেখে।

Solution:

Django filename change করে।

```
style.css
→ style.a4c8f3.css
```

Browser নতুন file download করে।

এটাকে বলে **cache busting**।

5️⃣ Alternative Storage

```
STATICFILES_STORAGE = "whitenoise.storage.CompressedStaticFilesStorage"
```

এটা শুধু compression করে।

কিন্তু

❌ cache busting করে না

Production এ normally use করা হয়:

```
CompressedManifestStaticFilesStorage
```

6️⃣ WhiteNoise development mode

```
INSTALLED_APPS = [
    "whitenoise.runserver_nostatic",
    "django.contrib.staticfiles",
]
```

এটা development server এ static serve করার behaviour change করে।

Normally:

```
django runserver
```

নিজেই static serve করে।

WhiteNoise enable করলে:

```
runserver → WhiteNoise static serve করবে
```

এতে production behaviour simulate হয়।

7️⃣ `collectstatic`

```
python manage.py collectstatic
```

এটা run করলে:

✔ সব app এর static file collect হয়
✔ compression হয়
✔ hashed filename generate হয়

Example:

```
staticfiles/
│
├── css/style.3afc12.css
├── css/style.3afc12.css.gz
├── js/app.92fa33.js
└── js/app.92fa33.js.gz
```

8️⃣ `STATICFILES_DIRS`

```
STATICFILES_DIRS = [BASE_DIR / "static_build"]
```

এটা external static folder add করে।

Example:

```
project/
│
├── static_build/
│   ├── main.js
│   └── style.css
│
├── apps/
```

এগুলো collectstatic এ include হবে।

Real use case

Frontend build tool:

- **Webpack**
- **Browserify**
- **Vite**

build করলে output আসে:

```
static_build/
   main.js
   style.css
```

Django collectstatic এগুলো collect করে।

9️⃣ Complete production setup example

```
STATIC_URL = "/static/"

STATIC_ROOT = BASE_DIR / "staticfiles"

STATICFILES_DIRS = [
    BASE_DIR / "static_build"
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
]

STORAGES = {
    "staticfiles": {
        "BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}
```

🔟 Production workflow

Deployment এ normally flow হয়:

```
git pull
↓
pip install
↓
python manage.py migrate
↓
python manage.py collectstatic
↓
run server
```

🔥 Industry tip

Big companies normally use:

```
Django + S3 + CloudFront
```

instead of WhiteNoise.

Tools:

- **Amazon S3**
- **Amazon CloudFront**

Static file CDN দিয়ে serve করে।

Django production-এ **static files (CSS, JS, images)** serve করার জন্য সাধারণত ৩টা architecture ব্যবহার করা হয়:

1️⃣ **WhiteNoise**
2️⃣ **Nginx**
3️⃣ **S3 + CDN**

আমি industry level architecture diagram সহ explain করছি। 👇

1️⃣ WhiteNoise Architecture (Simple Deployment)

এটা ছোট বা medium project এর জন্য খুব popular।

Flow

```
User Browser
      |
      ▼
Django Server
(Gunicorn + WhiteNoise)
      |
      ▼
Static Files (staticfiles/)
```

Tools

- **Django**
- **WhiteNoise**
- **Gunicorn**

Example configuration

```
STATIC_ROOT = BASE_DIR / "staticfiles"

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
]

STORAGES = {
    "staticfiles": {
        "BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}
```

Deployment flow

```
git pull
pip install
python manage.py migrate
python manage.py collectstatic
gunicorn config.wsgi
```

Advantages

✔ Setup easy
✔ Nginx লাগেনা
✔ small server এ চলে

Disadvantages

❌ High traffic এ slow হতে পারে
❌ CDN support নেই

2️⃣ Nginx Static Architecture (Most Common)

Industry তে অনেক project এই architecture use করে।

Flow

```
User Browser
      |
      ▼
Nginx
 |        |
 |        └──────► Static Files
 |                /var/www/static/
 |
 ▼
Gunicorn
 |
 ▼
Django
```

Tools

- **Nginx**
- **Gunicorn**
- **Django**

Static request example

User request:

```
/static/css/style.css
```

Nginx directly serve করে:

```
/var/www/static/css/style.css
```

Django এর কাছে request যায় না।

Nginx config example

```
location /static/ {
    alias /var/www/static/;
}
```

Advantages

✔ Very fast
✔ Django load কমে
✔ production ready

Disadvantages

❌ extra configuration
❌ server management লাগে

3️⃣ S3 + CDN Architecture (Large Scale)

Large scale SaaS system এ এটা use হয়।

Flow

```
User Browser
      |
      ▼
CDN (CloudFront)
      |
      ▼
S3 Bucket
(static files)

Django API Server
(Gunicorn)
```

Tools

- **Amazon S3**
- **Amazon CloudFront**
- **Django**

Static URL example

```
https://cdn.example.com/css/style.css
```

File actually stored in:

```
S3 bucket
```

Django configuration

Normally use:

- **django-storages**

Example:

```
STATICFILES_STORAGE = "storages.backends.s3boto3.S3Boto3Storage"

STATIC_URL = "https://cdn.example.com/"
```

Advantages

✔ Global CDN speed
✔ scalable
✔ server load almost zero

Disadvantages

❌ AWS cost
❌ setup complex

4️⃣ Architecture Comparison

Feature	WhiteNoise	Nginx	S3 + CDN
Setup difficulty	Easy	Medium	Hard
Performance	Medium	High	Very High
Scalability	Low	Medium	Very High
Cost	Low	Low	Medium
Use case	small apps	production apps	large systems

5️⃣ Real Industry Usage

Typical architectures:

Small SaaS

```
Django + Gunicorn + WhiteNoise
```

Medium Startup

```
Nginx
   |
Gunicorn
   |
Django
```

Large System

```
CloudFront CDN
      |
S3 static storage

API Servers
  └── Django + Gunicorn
```

6️⃣ Example: AI CCTV System (তোমার project)

তোমার **AI CCTV system** এর জন্য recommended architecture:

```
React Frontend
      |
      ▼
Nginx
 |        |
 |        └── Static files
 |
 ▼
Django API
```

```
  |
  ▼
AI Workers (Celery)
  |
  ▼
Redis
```

Tools:

- **Django**
- **React**
- **Redis**
- **Celery**

💡 চাইলে আমি তোমাকে আরও একটা important জিনিস দেখাতে পারি:

Complete Django production architecture (industry level)

```
React
   |
Nginx
   |
Django API
   |
PostgreSQL
   |
Redis
   |
Celery workers
   |
AI inference service
```

এটা বুঝলে তুমি **production backend system design expert level clear বুঝবে।**